

64-56 Projet de développement sur mandat

Manuel du développeur

2023 - 2024

Groupe « Les Dat'Accros-bates »

Christophe Alexakis

Florian Naef

Pamela Verdon

Faton Zumeri

HEG – Haute Ecole de Gestion

Genève

Table des changements

Version	Date	Auteur / Autrice	Sous-titres / Chapitres	Description
V0	04.05.2024	Pamela		Version originale
V1	05.05.2024	Pamela	Vue d'ensemble du projet	Ajout du contenu
V1	05.05.2024	Pamela	Développement Frontend → Interaction avec Firestore	Ajout du contenu
V1	16.05.2024	Christophe	Développement Frontend → Technologies utilisées	Précisions de certaines dépendances
V1	16.05.2024	Christophe	Développement Frontend → Architecture et Interaction des Composants Frontend → Schéma des composants	Ajout de la section et création du tableau
V1	17.05.2024	Pamela	Développement Backend → Authentification et Autorisation	Ajout du contenu
V1	17.05.2024	Pamela	Interaction Avec la Deuxième Application	Ajout de la section et du contenu
V1	26.05.2024	Pamela	Liste des annexes	Ajout d'une liste recensant toutes les annexes mentionnées dans le document

Table des matières

Introduction	3
Vue d'ensemble du Projet	4
Description de l'architecture	4
Technologies utilisées et leurs rôles	4
Schéma général de l'application	5
Configuration de l'Environnement de Développement	6
Structure du Code	7
Organisation des fichiers et dossiers	7
Conventions de nommage	8
Modules et composants principaux	8
Développement Frontend	9
Technologies utilisées	9
Architecture et Interaction des Composants Frontend	10
Structure CSS et Mise en Page	10
Routage et Navigation	11
Schéma des composants	11
Communication entre les Composants	12
Gestion des <i>Hooks</i>	12
Intégration de React Query & React Query Devtools	13
Interaction avec Firestore	13
Développement Backend	18
Firebase	18
Authentification et Autorisation	18
Authentification	18
Gestion des Utilisateurs	18
Modèle d'E-mail de Réinitialisation :	19
Autorisations	19
Rôles et Privilèges	19
Modèles de données	20
Schéma de la base de données	21
Interaction Avec la Deuxième Application	23
Import	23
Export	24
Ressources Additionnelles	25
Liste des annexes	26

Introduction

Cette ressource est spécialement conçue pour vous guider dans le développement, le déploiement et la maintenance de notre application, conçue pour répondre aux besoins spécifiques de la Commission Agrès de Genève.

Notre application vise à simplifier la gestion des données des gymnastes participant aux concours de la Commission Agrès de Genève. Elle permet aux administrateurs et aux coachs d'enregistrer, de suivre et de mettre à jour les informations pertinentes concernant les gymnastes et leurs résultats aux concours.

Par ailleurs, une deuxième application a été mise en place par une autre équipe, avec laquelle notre application peut interagir à l'aide d'import et d'export de fichiers CSV.

Développée dans un contexte agile et collaboratif, cette application repose sur des technologies modernes telles que Firebase pour le backend, React pour le frontend et React Query pour la gestion des requêtes et du cache.

Ce manuel du développeur a pour objectif de fournir aux membres de l'équipe de développement toutes les informations nécessaires pour contribuer efficacement au projet. Vous y trouverez des instructions détaillées sur la configuration de l'environnement de développement, la structure du code, les bonnes pratiques de développement, la sécurité, les tests, le déploiement et la maintenance de l'application.

Vue d'ensemble du Projet

Description de l'architecture

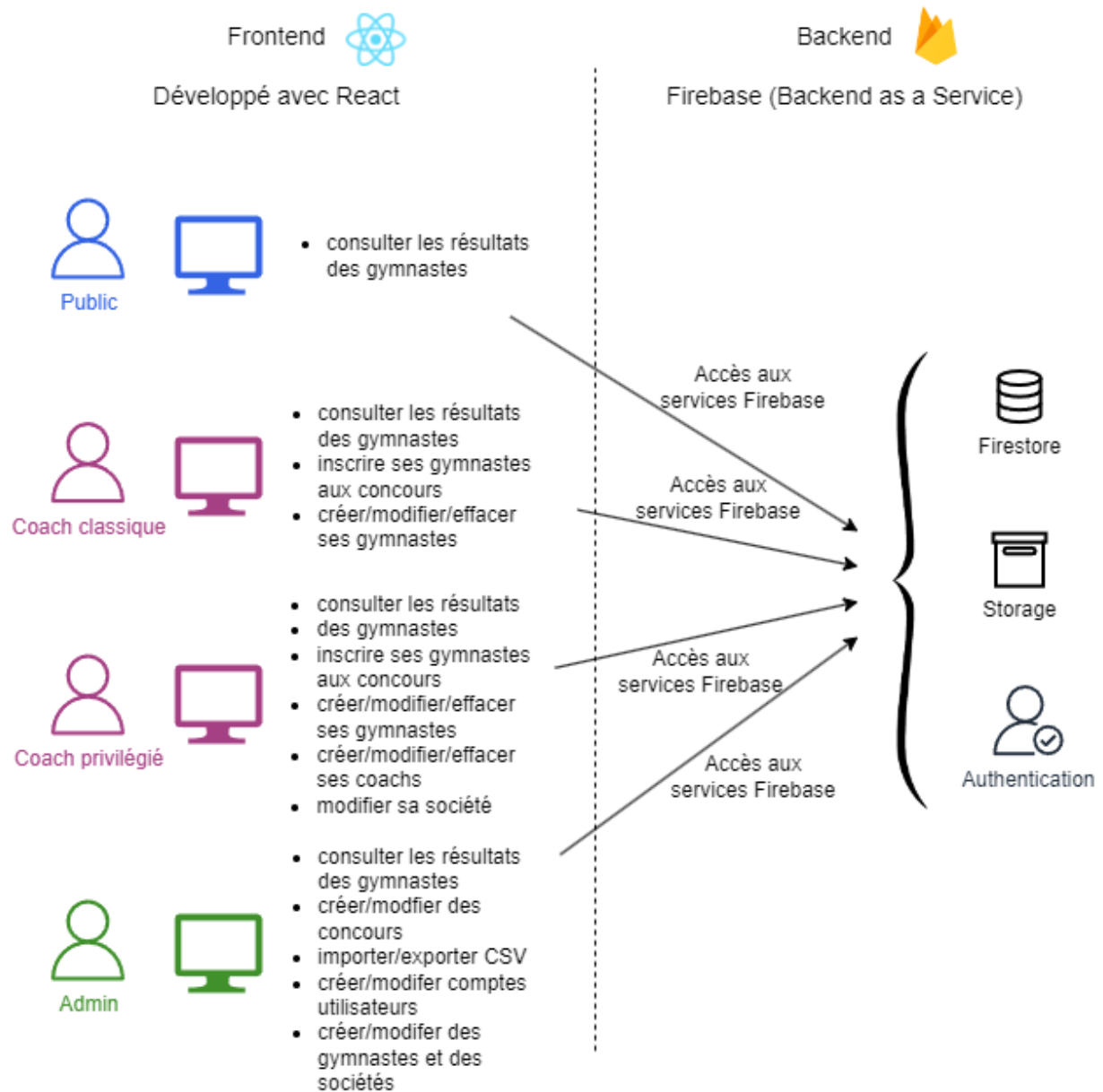
L'architecture de l'application est conçue autour de deux composants principaux : le frontend et le backend. Le frontend, développé avec React, se concentre sur l'interface utilisateur, tandis que le backend, basé sur Node.js, assure la logique métier et l'accès aux services de Firebase, une plateforme BaaS (Backend as a Service) qui fournit une gamme de fonctionnalités telles que l'authentification des utilisateurs, le stockage de fichiers, des base de données, etc.

Technologies utilisées et leurs rôles

Les technologies clés utilisées incluent React pour l'interface utilisateur, Firebase pour les services BaaS, Node.js pour le backend, et Vite pour le développement rapide.

- React : Utilisé pour créer une interface utilisateur interactive et réactive, offrant une expérience utilisateur fluide.
- Firebase : Fournit une base de données cloud pour stocker et gérer les données de l'application, ainsi que des services d'authentification pour gérer l'identification des utilisateurs.
- Node.js : Utilisé de manière transparente dans le backend de l'application pour faciliter l'exécution de la logique métier et l'interaction avec Firebase. Bien qu'il soit présent dans l'architecture de l'application, son rôle est plus implicite, agissant comme une couche sous-jacente pour assurer le bon fonctionnement du backend sans nécessiter une configuration ou un développement spécifique.
- Vite : Outil de développement rapide permettant une configuration simplifiée et un rechargement instantané des modifications apportées au code frontend.

Schéma général de l'application



Configuration de l'Environnement de Développement

Pour configurer l'environnement de développement, suivez ces instructions :

1. Installation de Node.js : Assurez-vous d'avoir Node.js installé sur votre machine. Vous pouvez le télécharger à partir du site officiel de Node.js et suivre les instructions d'installation (voir "Ressources Additionnelles"). Node.js est nécessaire pour exécuter les scripts npm utilisés dans le développement de l'application.
2. Configuration de l'Environnement Frontend : Notre application utilise React.js comme framework frontend. Vous devrez installer les dépendances nécessaires en exécutant *npm install* dans le répertoire du projet. Une fois les dépendances installées, vous pouvez démarrer le serveur de développement en exécutant *npm run dev*.
3. Configuration de Firebase :
 - Assurez-vous que le projet Firebase correspond aux informations fournies dans le fichier "firebaseConfig.jsx". Vous pouvez trouver ces informations dans les paramètres de votre projet Firebase.
 - Pour Firebase Storage, veuillez vous assurer qu'il existe un dossier nommé "logos" pour stocker les logos des sociétés.
 - Pour Firestore, il n'y a pas besoin de préalablement créer des collections et documents dans Firestore. Assurez-vous simplement que la collection "cag_utilisateurs" existe, contenant au moins les données de l'administrateur initial de l'application. Les règles de sécurité sont configurées pour autoriser uniquement les utilisateurs authentifiés à interagir avec les données. Vous pouvez trouver et modifier ces règles dans la section "Cloud Database" de la console Firebase, sous l'onglet "Règles". Voici les règles à mettre en place :

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    // Autoriser la lecture et l'écriture sans authentification pour les tests
    match /{document=**} {
      allow read, write;
    }

    // Autoriser la lecture et l'écriture pour les utilisateurs authentifiés
    match /cag_utilisateurs/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;

      // Autoriser l'accès en lecture à tous les documents dans la sous-collection "societes" des utilisateurs authentifiés
      match /societes/{document=**} {
        allow read: if request.auth != null;
      }
    }
  }
}
```

Ces étapes devraient vous permettre de configurer correctement votre environnement de développement pour travailler sur l'application. N'oubliez pas de consulter également le README du projet pour des instructions détaillées sur les dépendances à installer et sur l'exécution de l'application.

Dans le README, vous trouverez également les informations d'authentification de trois types de comptes : coach classique, coach privilégié et administrateur afin d'accéder à l'application avec le rôle souhaité.

Structure du Code

Organisation des fichiers et dossiers

Le dossier racine de notre projet, nommé "PSM_DatAccrosBates" est décomposé comme suit :

 node-modules

 public

 src

Fichiers de configuration (package.json, index.html, etc)

Le fichier "src" ci-dessus est décomposé de la façon suivante :

 assets

 components : Comprend tous les composants réutilisables, tous classés dans des sous-dossiers

 ... : 14 sous-dossiers contenant les composants appelés par les pages principales

 data : Contient les fichiers de configuration nécessaires à l'application, les fichiers d'import de données, ainsi que les méthodes React Query

 import : Contient les fichiers nécessaires à l'importation des données dans Firestore

Autres fichiers

 hooks : Rassemble les hooks personnalisés utilisés dans l'application

 pages : Contient les pages principales de l'application, chacune appelant les composants nécessaires

 pagesOnglets : Contient le composant parent des pages avec onglets

Autres fichiers

 readme

 tests : Comprend les fichiers de tests unitaires et de tests d'intégration pour l'application

App.css : Fichier de styles global pour l'application

App.jsx : Point d'entrée principal de l'application où est définie la structure globale

AppRoutes.jsx : Fichier contenant la configuration des routes de l'application

main.jsx : Fichier principal où l'application est initialisée et montée dans le DOM

Conventions de nommage

Les fichiers et dossiers sont nommés de manière descriptive et cohérente pour faciliter la compréhension et la navigation.

Pour les noms de fichiers, de composants et de méthodes, nous utilisons la convention *camelCase*. De même, les fichiers CSS sont nommés de manière similaire à leurs homologues JavaScript, tandis que les noms de classe dans ces fichiers adoptent la convention *kebab-case*.

Les fichiers JavaScript commencent par une majuscule, alors que les fichiers CSS et les noms de dossiers commencent par une minuscule.

Modules et composants principaux

Les principaux modules et composants sont les suivants :

📁 `src/App.jsx` : Point d'entrée principal de l'application où est définie la structure globale

📁 `src/AppRoutes.jsx` : Fichier contenant la configuration des routes de l'application

📁 `src/main.jsx` : Fichier principal où l'application est initialisée et montée dans le DOM

📁 `src/data/firebaseConfig.jsx` : Fichier contenant la configuration de Firebase, y compris l'initialisation et l'exportation des instances de Firestore, d'authentification et du storage

📁 `src/data/Data.jsx` : Fichier qui contient des méthodes pour interagir avec la base de données Firestore, telles que des requêtes React Query ou des fonctions pour lire ou écrire des données dans Firestore

📁 `src/hooks/authContext.jsx` : Ce fichier définit un contexte d'authentification React qui fournit des informations sur l'utilisateur connecté et ses sociétés associées. Il utilise Firebase Authentication pour gérer l'authentification de l'utilisateur et Firestore pour récupérer les données utilisateur à partir de la base de données

Technologies utilisées

React : Nous avons choisi React comme bibliothèque principale pour la construction de l'interface utilisateur de notre application en raison de sa popularité, de sa flexibilité et de sa richesse en fonctionnalités.

React Query : Nous avons intégré React Query pour la gestion des requêtes et des données côté client. Cette bibliothèque fournit des *hooks* qui facilitent l'exécution de requêtes de données et la gestion de leur mise en cache, ce qui améliore les performances et l'expérience utilisateur de notre application.

React-Icons : Pour l'intégration d'icônes dans l'application, nous avons utilisé la bibliothèque React-Icons, car elle propose une grande sélection d'icônes prêtes à l'emploi. En utilisant React-icons, nous avons pu incorporer facilement des icônes dans divers composants, ce qui rend l'application plus intuitive.

Material-UI et MUI : Ces bibliothèques offrent une gamme de composants pré-conçus basés sur les directives de conception de Material Design. Elles ont été utilisées spécifiquement pour la création des tableaux dans la page d'inscription et pour les "accordions" également dans la page d'inscription. Ces bibliothèques offrent directement de petites animations comme l'ouverture et la fermeture des accordions.

Nous avons également utilisé d'autres dépendances pour des fonctionnalités spécifiques telles que la création de modales interactives, la recherche avancée et le rendu de contenu Markdown dans notre application. Ces dépendances contribuent à rendre notre application plus conviviale.

En complément, nous avons développé une grande partie du style CSS en interne, plutôt que de recourir à un grand nombre de bibliothèques externes. Ainsi, la plupart des composants avec leurs styles créés sont regroupés dans le dossier  src/components. Cette approche permet un contrôle total sur l'apparence et le comportement des composants, tout en garantissant une plus grande flexibilité dans la conception de l'application.

Architecture et Interaction des Composants Frontend

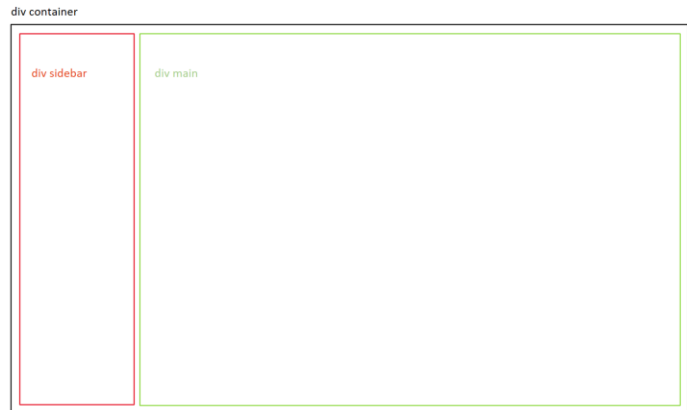
Structure CSS et Mise en Page

Dans la structure CSS et la mise en page de notre application, nous avons adopté une approche basée sur Grid et Flexbox pour obtenir une mise en page fluide et flexible. Nous avons utilisé un système de grille 12x12, où la page entière est représentée par une div container subdivisée en 12 colonnes et 12 lignes. Cette division nous permet une grande flexibilité dans le positionnement des éléments.

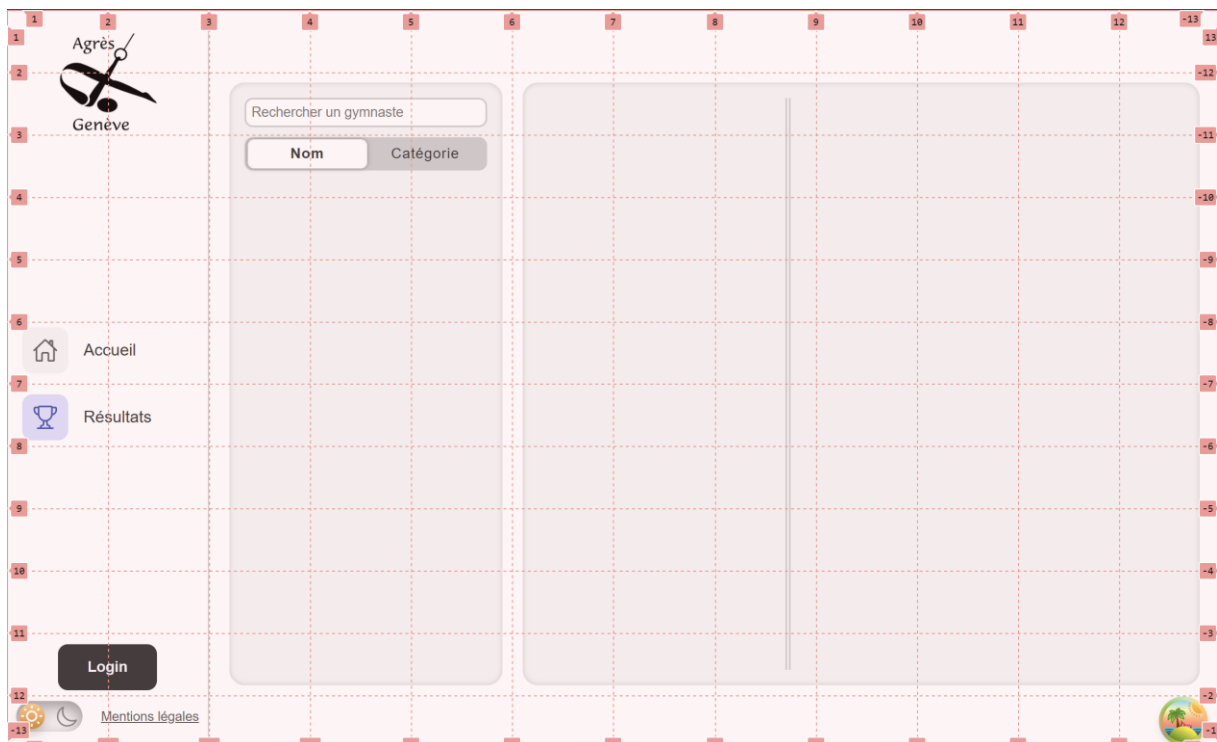
La barre latérale (sidebar) et la section principale (main) de la page sont organisées à l'aide d'un *subgrid*. La barre latérale occupe 2 colonnes et 12 lignes, tandis que la section principale occupe les dix colonnes restantes et 12 lignes. Cette subdivision nous permet de définir clairement les zones de la page et de les organiser de manière efficace.

Pour visualiser cette structure en action, nous avons inclus un exemple d'écran montrant l'emboîtement de la *div container*, de la barre latérale et de la section principale, démontrant ainsi la logique et la cohérence de notre mise en page.

Le code CSS de nos 3 principales *div* se trouve dans le fichier `src/App.css`. Ici se trouve également le CSS des "box". Ce sont les éléments principaux que nous avons placés dans notre *div main* grâce à ce système de grille. Puis, à l'intérieur de ces "box", nous avons utilisé Flexbox pour le positionnement et l'alignement.



L'exemple ci-dessous montre la grille utilisée et les "box" que nous retrouvons dans toute notre application. A noter que seule la page des inscriptions n'utilise pas de box mais directement des tableaux de la bibliothèque Material-UI.



Routage et Navigation

Le routage dans l'application est géré par React Router. Dans le fichier `src/AppRoutes.jsx`, nous avons configuré les différentes routes de notre application en utilisant les composants *Routes* et *Route* fournis par React Router.

Dans la configuration de routage, chaque route est associée à un composant spécifique qui sera rendu lorsque l'URL correspondante sera atteinte. Par exemple, la route `/accueil` est associée au composant *Accueil*, la route `/resultats` est associée au composant *Resultats*, et ainsi de suite.

La navigation entre les différentes pages de l'application se fait en changeant l'URL dans la barre d'adresse du navigateur ou en cliquant sur des liens qui correspondent à des routes spécifiques. Lorsqu'un utilisateur accède à une URL correspondant à une route définie dans la configuration de routage, le composant associé à cette route est rendu à l'écran, permettant ainsi à l'utilisateur de naviguer dans l'application.

En fonction du rôle de l'utilisateur, certaines routes de l'application peuvent être accessibles ou non. Par exemple, les administrateurs peuvent accéder à des routes spécifiques telles que la création ou la modification de concours, de coaches, d'administrateurs, de gymnastes ou de sociétés. En revanche, les utilisateurs non connectés ou les utilisateurs avec des rôles différents n'ont pas accès à ces fonctionnalités et sont redirigés vers d'autres pages de l'application ou vers la page 404 par défaut.

Schéma des composants

Ce tableau récapitule les principaux composants utilisés par les pages de notre application. Chaque ligne précise le nom d'un ou plusieurs composants, qu'est-ce qu'ils contiennent (leurs fonctionnalités ou d'autres composants) et dans quels composants ils sont utilisés.

Composant	Contenu	Utilisation
Formulaires (Admin, Coach, Gymnaste, Concours, Société)	BoutonForm Message Carte Bouton Switch CheckBox	Les composants des Admin, Coach, Gymnaste, Concours, Société – Créer/Modifier
BoutonForm	Un bouton avec style CSS personnalisé	Tous les formulaires
Carte	Une carte/badge pour représenter le gymnaste lors de sa création	Tous les formulaires sauf concours
Message	Une <i>div</i> , avec une icône, pour informer l'utilisateur que tous les champs sont remplis ou non	Tous les formulaires
BoutonSwitch	Un bouton switch avec un texte personnalisable	Formulaires Gymnaste (Homme, Femme) Formulaires Coach (Classique, privilégié) Formulaire Concours (Equipe, individuel)
CheckBox	Une liste transformée en liste de checkBox	Formulaires Concours (catégories autorisées) Formulaires Coach (sociétés)
Calendrier	Un calendrier interactif	Formulaires de concours, pour choisir la date du concours et la date limite d'inscription
ConcoursListe, GymnasteListe, ResultatsListe, SocieteListe, UtilisateurListe	Des cartes adaptées en fonction du contenu affiché	Tous les composants de modifications, Résultats (liste concours)

Communication entre les Composants

Dans notre application, la communication entre les composants se fait principalement via deux mécanismes : la transmission des données via les *props* dans React et l'utilisation des contextes pour partager des données entre les composants.

- Transmission des données via les *props* : Les *props* sont utilisées pour transmettre des données et des fonctions entre les composants parents et enfants. Concrètement, cela signifie que le composant parent peut passer des informations spécifiques à ses enfants en utilisant les *props*, qui sont essentiellement des attributs ou des paramètres transmis lors de l'invocation d'un composant enfant. Ces données peuvent inclure des valeurs telles que des chaînes de caractères, des nombres, des fonctions, ou même des objets entiers, permettant ainsi de contrôler le comportement et l'apparence des composants enfants en fonction du contexte dans lequel ils sont utilisés.
- Utilisation des contextes pour partager des données : Nous avons intégré un contexte pour gérer de manière centralisée des données partagées entre plusieurs composants sans avoir à passer par les *props* à chaque niveau de l'arborescence des composants. Ainsi, le contexte "AuthProvider" est utilisé pour gérer l'authentification des utilisateurs. Ce contexte contient des informations sur l'utilisateur connecté, telles que son ID, son rôle, etc.

Gestion des Hooks

Comme expliqué ci-dessus, nous avons utilisé un contexte pour gérer l'authentification des utilisateurs dans notre application. Le composant de contexte "AuthProvider" enveloppe toute l'application, ce qui permet d'accéder facilement aux informations sur l'utilisateur connecté à partir de n'importe quel composant de l'application. Ce contexte fournit des méthodes pour l'authentification, telles que la connexion, la déconnexion et la vérification de l'état de l'utilisateur connecté.

De plus, nous avons utilisé le package React Query (TanStackQuery) pour gérer les requêtes et les mutations de données dans notre application. Ce package fournit des *hooks* tels que *useQuery* et *useMutation* qui facilitent la récupération et la mise à jour des données Firestore, tout en gérant automatiquement le cache et les recharges de données. En utilisant React Query, les performances sont améliorées en évitant les requêtes redondantes au serveur.

Intégration de React Query & React Query Devtools

Pour la gestion du cache avec React Query, chaque résultat de requête est conservé dans le cache de façon infinie jusqu'à ce qu'un rafraîchissement ou la fermeture de l'application soit effectué. Ainsi, cela permet de diminuer les appels à Firebase. De plus, à chaque changement (mutation), la requête (query) correspondante est effacée du cache. Ainsi, dès qu'elle est de nouveau nécessaire, elle sera rappelée avec les données mises à jour, éliminant ainsi le besoin de rafraîchissement ou d'autres actions pour être mis à jour.

Pour afficher les outils de développement React Query dans votre application, vous pouvez suivre ces étapes :

1. Assurez-vous que "@tanstack/react-query-devtools" est installé dans votre projet.
2. Ajoutez le composant "<ReactQueryDevtools />" à votre arborescence de composants, par exemple ici, dans le fichier `src/App.jsx` :

```
<div className="main">
  <AppRoutes />
  <ReactQueryDevtools />
</div>
```

3. Appuyez sur cette icône  désormais visible depuis n'importe quelle page de l'application.

Cet outil permet de visualiser les requêtes effectuées, de vérifier leur état (fresh, stale, etc.), et de voir les données associées à chaque requête. De plus, il permet de distinguer les requêtes actives des requêtes inactives, offrant ainsi un aperçu complet de l'état des requêtes dans l'application. Enfin, il est important de noter qu'à chaque fois que l'application est rafraîchie ou fermée, les données stockées dans le cache via React Query sont effacées, garantissant ainsi la fraîcheur des données lors des prochaines utilisations de l'application.

Interaction avec Firestore

Pour interagir avec Firestore dans notre application frontend, nous utilisons le SDK Firebase, qui fournit une interface conviviale pour communiquer avec la base de données Firestore à partir du code JavaScript.

Le SDK Firebase offre une gamme complète de fonctionnalités pour faciliter la gestion des données, y compris l'initialisation de l'application avec les paramètres de configuration Firebase requis, l'établissement d'une connexion à Firestore, et l'utilisation de méthodes simples pour effectuer des opérations telles que l'ajout, la modification, la récupération et la suppression de données.

Certains des appels à Firestore sont effectués en utilisant le *hook* React Query, ce qui nous permet de gérer efficacement le cache des données et d'optimiser les performances de notre application. Les méthodes interagissant avec Firebase se trouvent dans le fichier `src/data/Data.jsx`. Voici un tableau résumant les méthodes, leur but, leur valeur de retour et leur comportement en cas de succès et en cas d'erreur.

Nom de la méthode	But	Retour	Comportement succès	Comportement erreur
Récupération des données de Firestore (React Query)				
useAllGymnastesData	Récupérer toutes les données des gymnastes de Firestore	Tableau d'objets de gymnastes		
useParticipationData	Récupérer les participations d'un gymnaste spécifique	Tableau d'objets des participations ainsi que les concours associés		
useAnciensGymnastes	Récupérer les gymnastes déjà existants mais non associés à une société	Tableau d'objets de gymnastes		
useGymnastesSocieteData	Récupérer les gymnastes associés à une société spécifique	Tableau d'objets de gymnastes		
useSocieteData	Récupérer toutes les sociétés	Tableau d'objets de sociétés		
useConcoursData	Récupérer tous les concours de la saison en cours	Tableau d'objets de concours		
useAdminData	Récupérer tous les administrateurs	Tableau d'objets d'administrateurs		
useCoachData	Récupérer tous les coaches	Tableau d'objets de coaches		
useCoachSocieteData	Récupérer les coaches associés à une société spécifique	Tableau d'objets de coaches		
useInscriptionData	Récupérer les gymnastes déjà inscrits à un concours pour une société donnée	Tableau d'objets de gymnastes		

Nom de la méthode	But	Retour	Comportement succès	Comportement erreur
Ajout de données dans Firestore (React Query)				
useAddGymnaste	Ajouter un gymnaste à la collection "cag_gymnaste" et ajoute une référence de ce gymnaste dans la liste de gymnastes de sa société		Ajoute le gymnaste à la base de données Firestore et efface les requêtes de gymnastes désormais obsolètes	Affiche un message d'erreur dans la console
useAddCoach	Ajouter un coach à la collection "cag_utilisateurs" et lui créer un compte		Ajoute le coach dans Authentication et Firestore et efface les requêtes de coachs désormais obsolètes	Affiche un message d'erreur dans la console
useAddAdmin	Ajouter un administrateur à la collection "cag_utilisateurs" et lui créer un compte		Ajoute l'admin dans Authentication et Firestore et efface les requêtes d'admin désormais obsolètes	Affiche un message d'erreur dans la console
useAddSociete	Ajouter une société à la collection "cag_societe"		Ajoute la société à Firestore et son logo dans Storage et efface les requêtes de sociétés désormais obsolètes	Affiche un message d'erreur dans la console
useAddConcours	Ajouter un concours à la collection "cag_concours"		Ajoute le concours à la base de données Firestore et efface les requêtes de concours désormais obsolètes	Affiche un message d'erreur dans la console
useAddInscription	Ajouter des gymnastes à la liste des inscriptions pour un concours et une société donnée		Ajoute des gymnastes à la liste des inscriptions et efface les requêtes d'inscriptions désormais obsolètes	Affiche un message d'erreur dans la console

Nom de la méthode	But	Retour	Comportement succès	Comportement erreur
Modification des données de Firestore (React Query)				
useSetGymnaste	Modifier les données d'un gymnaste dans la collection "cag_gymnaste" et met à jour sa référence dans la liste de gymnastes de sa société		Met à jour les données du gymnaste dans Firestore et efface les requêtes de gymnastes désormais obsolètes	Affiche un message d'erreur dans la console
useDeleteGymnaste	Dissocier un gymnaste d'une société dans la collection "cag_gymnaste" et "cag_societe"		Dissocie le gymnaste de la société dans Firestore et efface les requêtes de gymnastes désormais obsolètes	Affiche un message d'erreur dans la console
useDeleteCoach	Dissocier un coach d'une société dans la collection "cag_utilisateurs"		Dissocie le coach de la société dans Firestore et efface les requêtes de coaches désormais obsolètes	Affiche un message d'erreur dans la console
useSetCoach	Modifier les données d'un coach dans la collection "cag_utilisateurs"		Met à jour les données du coach dans Firestore et efface les requêtes de coaches désormais obsolètes	Affiche un message d'erreur dans la console
useSetAdmin	Modifier les données d'un administrateur dans la collection "cag_utilisateurs"		Met à jour les données de l'administrateur dans Firestore et efface les requêtes d'admin désormais obsolètes	Affiche un message d'erreur dans la console
useSetSociete	Modifier les données d'une société dans la collection "cag_societe" et son logo		Met à jour les données de la société dans Firestore, ainsi que son logo dans Storage et efface les requêtes de sociétés désormais obsolètes	Affiche un message d'erreur dans la console
useSetConcours	Modifier les données d'un concours dans la collection "cag_concours"		Met à jour les données du concours dans Firestore et efface les requêtes de concours désormais obsolètes	Affiche un message d'erreur dans la console

Nom de la méthode	But	Retour	Comportement succès	Comportement erreur
Import et Export des fichiers CSV dans Firestore				
exportCSV	Exporter les données des gymnastes inscrits à un concours au format CSV		Génère un fichier CSV téléchargeable contenant les données des gymnastes inscrits au concours	
importCSV	Importer les résultats des gymnastes à un concours à partir d'un fichier CSV		Met à jour la sous-collection "participation" de chaque gymnaste avec les résultats et efface les requêtes désormais obsolètes	Affiche un message d'erreur dans la console

Développement Backend

Firebase

Dans le développement de l'application, Firebase a été notre choix principal pour le backend, fournissant une solution complète sans serveur pour la gestion des fonctionnalités backend. Firebase offre une gamme de services essentiels pour le développement rapide d'applications web et mobiles, éliminant la nécessité de configurer et de gérer une infrastructure serveur complexe.

Nous avons intégré Firebase pour répondre à nos besoins backend. Firestore, notre base de données NoSQL, garantit une gestion efficace et flexible des données de notre application. Firebase Authentication gère l'authentification des utilisateurs de manière sécurisée, tandis que Firebase Storage assure le stockage des fichiers et des médias.

Contrairement à l'utilisation traditionnelle d'un framework backend, nous avons opté pour Firebase en tant que plateforme sans serveur. Cette approche nous a permis de nous concentrer sur le développement des fonctionnalités frontend de l'application, en laissant Firebase gérer les aspects complexes du backend.

Authentification et Autorisation

L'application utilise Firebase pour gérer à la fois l'authentification et l'autorisation des utilisateurs, garantissant ainsi la sécurité et la gestion des accès aux différentes fonctionnalités de l'application. L'authentification est gérée grâce à Firebase Authentication et les autorisations sont définies dans la base de données Firestore.

Authentification

Processus de création de compte :

- Admins : Les administrateurs peuvent créer des comptes pour d'autres administrateurs et des coachs.
- Coachs Privilégiés : Ils peuvent créer des comptes pour des coachs qui sont reliés à leur société.
- Lors de la création d'un compte, un mot de passe est généré automatiquement et peut être affiché à la personne qui remplit le formulaire.
- Un e-mail de réinitialisation de mot de passe est envoyé au nouvel utilisateur pour des raisons de sécurité, afin qu'il puisse définir un mot de passe personnel qu'il est le seul à connaître. Ce processus garantit que l'utilisateur puisse changer le mot de passe initial automatiquement généré. Ce même e-mail est utilisé pour la réinitialisation en cas de mot de passe oublié.

Gestion des Utilisateurs

- Tous les comptes actifs peuvent être visualisés et gérés dans la console Firebase sous Authentication -> Utilisateurs.
- Ici, les administrateurs peuvent voir tous les e-mails des comptes actifs, supprimer ou désactiver des comptes ou encore envoyer un e-mail de réinitialisation de mot de passe.
- Chaque utilisateur a un UID unique qui correspond à l'ID du document utilisateur correspondant dans la collection "cag_utilisateurs" de Firestore.

Modèle d'E-mail de Réinitialisation :

Le modèle d'e-mail utilisé pour la création d'un compte et la réinitialisation des mots de passe peut être trouvé et modifié dans Authentication -> Modèles -> Réinitialisation du mot de passe.

Autorisations

L'autorisation est basée sur les rôles définis pour chaque utilisateur dans la collection "cag_utilisateurs" de Firestore. Chaque document utilisateur possède un champ "role" qui détermine les privilèges d'accès de l'utilisateur.

Rôles et Privilèges

1. Public :

- Utilisateurs non connectés ou anonymes.
- Accès limité aux fonctionnalités de base et aux informations publiques.

2. Coach Classique :

- Utilisateurs connectés avec des privilèges de gestion des gymnastes et des inscriptions aux concours.
- Ne peuvent pas créer ou supprimer d'autres coaches.

3. Coach Privilégié :

- Privilèges étendus par rapport aux coaches classiques.
- Peuvent créer et supprimer des coaches reliés à leur société.
- Modification des informations de leur société.

4. Admin :

- Accès à toutes les fonctionnalités de gestion de l'application.
- Gestion complète des concours.
- Capacité de créer et gérer des gymnastes ainsi que des comptes pour d'autres administrateurs et coaches.
- Gèrent la relation avec la deuxième application, grâce à des fichiers CSV.
- Accès à des actions avancées directement dans la console Firebase.

Modèles de données

Ce modèle établit la structure conceptuelle des informations manipulées par l'application, identifiant les entités principales, leurs relations et les attributs associés. Il sert de guide pour le développement en fournissant une vue abstraite des données, indépendamment de leur stockage physique.

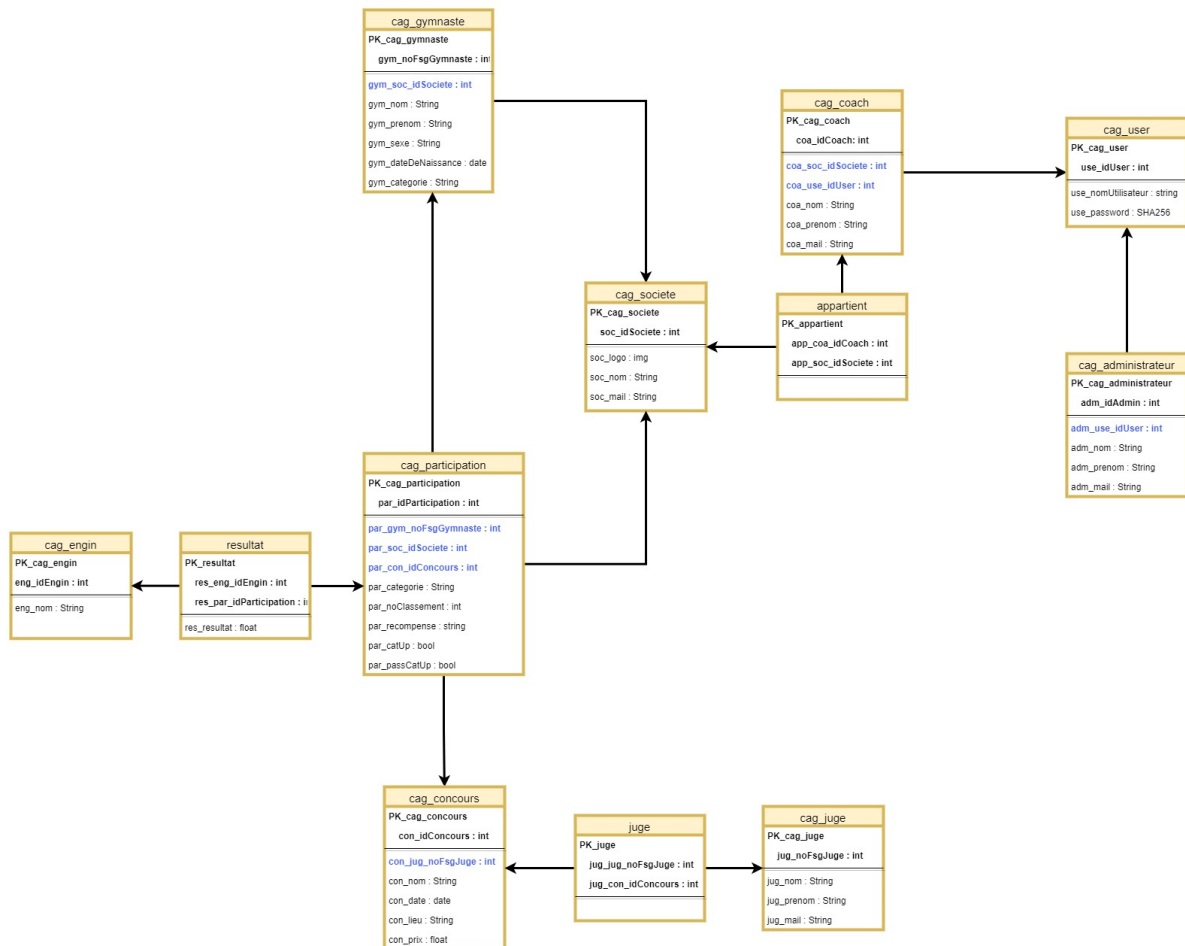


Schéma de la base de données

Dans Firestore, le système de gestion de base de données de Firebase, le modèle de données est traduit en un schéma concret. Ce schéma spécifie la structure physique des données, incluant les collections, les documents et les champs.

Collections	Documents	Champs	
cag_concours	id automatique	categories	(tableau de chaînes)
		date	(code temporel)
		dateInscription	(code temporel)
		equipe	(boolean)
		lieu	(chaîne)
		nom	(chaîne)
cag_gymnaste	id = noFSG	categorie	(chaîne)
		dateNaissance	(code temporel)
		idSociete	(référence)
		noFSG	(chaîne)
		nom	(chaîne)
		prenom	(chaîne)
		sexe	(chaîne)
		participation	(sous-collection)
participation	id automatique	categorie	(chaîne)
		classement	(chiffre)
		idConcours	(référence)
		passageCat	(boolean)
		recompense	(chaîne)
		resultats	(mappage)
		{	
		anneaux (chiffre)	
		bf (chiffre)	
		bp (chiffre)	
		saut (chiffre)	
		sol (chiffre)	
		}	
		equipe	(mappage)
		{	
		classement (chiffre)	
		no (chaîne)	
		total (chiffre)	
		} uniquement concours	
		équipe	

Collections	Documents	Champs	
cag_societe	id automatique	email	(chaîne)
		gymnastes	(tableau de références)
		logo	(chaîne)
		nom	(chaîne)
		inscription	(sous-collection)

inscription	id automatique	idConcours	référence
		gymnastesInscrits [	(tableau de mappages)
		{	
		catUp (boolean)	
		categorie (chaîne)	
		gymnaste (référence)	
		equipe (chiffre)	
		<i>uniquement pour</i>	
		<i>concours équipe</i>	
		}, ...]	

cag_utilisateurs	uid de l'authentification	email	(chaîne)
		nom	(chaîne)
		nomUtilisateur	(chaîne)
		prenom	(chaîne)
		role	(chaîne)
		societe <i>uniquement</i>	(tableau de références)
		<i>pour coaches</i>	

Interaction Avec la Deuxième Application

Comme expliqué dans l'introduction, une gestion de la communication est nécessaire avec la deuxième application conçue pour la Commission Agrès. Le système mis en place est l'import et l'export de fichiers CSV entre les applications. Nous parlons d'export lorsqu'il s'agit d'un fichier CSV généré dans notre application et destiné à l'autre application et nous parlons d'import lorsqu'il s'agit d'un fichier CSV à introduire dans notre application.

Un fichier CSV correspond à un concours spécifique.

Les méthodes "importCSV" et "exportCSV" ont été évoquées dans une section précédente (voir "Interaction avec Firestore"). Elles servent respectivement à introduire les résultats des gymnastes à un concours spécifique et à générer une liste des gymnastes inscrits par leurs coaches à un concours spécifique. La gestion du déroulement du concours se passe dans l'autre application.

Les exemples de fichiers CSV montrés ci-dessous sont **disponibles en annexe**.

Import

Pour que l'import soit fonctionnel, les fichiers CSV importés doivent suivre une configuration précise.

Voici un exemple pour un concours individuel :

	A	B	C	D	E	F	G	H	I	J	K	L
1	noFSG	nom	prenom	sol	anneaux	saut	bf	bp	noClassement	recompense	categorie	catUp
2		10214 Colin	Benjamin	9.95	8.7	9.15	9.00	9.75		3 bronze		7 false
3		10460 Vidal	Marcelle	7.50	8.55	9.20	9.55			6 distinction		4 true
4		10305 Gaudin	Tristan	8.4	8.75	9.20	3.3	8		6		4 false

La catégorie fait référence à la catégorie dans laquelle le gymnaste a concouru et qui ne correspond pas forcément à sa catégorie actuelle. Des champs peuvent être laissés vides, par exemple pour une femme, elle n'aura pas de résultat pour "bp" (barres parallèles). Aussi, tous les gymnastes ne reçoivent pas une récompense. Le nom et prénom n'ont pas d'utilité particulière dans notre méthode. Le champ "catUp" nous sert à savoir si le gymnaste a confirmé son passage de catégorie éventuel, et si tel est le cas, nous changeons sa catégorie dans Firestore.

Voici un exemple pour un concours par équipe :

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	noFSG	nom	prenom	sol	anneaux	saut	bf	bp	noClassement	recompense	categorie	catUp	noEquipe	classementEquipe	totalEquipe
2		10214 Colin	Benjamin	9.95	8.7	9.15	9.00	9.75		3 bronze		7 false	D1		3 147.8
3		10460 Vidal	Marcelle	7.50	8.55	9.20	9.55			6 distinction		4 true			
4		10305 Gaudin	Tristan	8.4	8.75	9.20	3.3	8		6		4 false	C3		1 150

Trois colonnes supplémentaires sont nécessaires pour préciser les résultats de l'équipe. Ils peuvent être laissés vides dans le cas où le gymnaste n'était pas dans une équipe.

Avec cette configuration, aucune vérification sur le concours sélectionné n'est possible. Il convient à l'administrateur de ne pas se tromper lorsqu'il choisit un concours auquel il veut importer un résultat.

Après le traitement du fichier, les résultats de chaque gymnaste sont ajoutés dans Firestore, dans une sous-collection nommée "participation", et sont visibles depuis la page "Résultats".

Export

Les fichiers CSV générés par notre application suivent également une configuration précise. Ils contiennent toutes les informations dont l'autre application a besoin pour être fonctionnelle. Ces informations sont récupérées depuis la sous-collection "inscription" des documents des sociétés.

Exemple pour un concours individuel :

	A	B	C	D	E	F	G	H
1	noFSG	nom	prenom	dateDeNaissance	sexe	categorie	catUp	societe
2	32247	Bouvet	Bernadette	16.03.1993	F		4 false	Versoix
3								

Exemple pour un concours par équipe :

	A	B	C	D	E	F	G	H	I
1	noFSG	nom	prenom	dateDeNaissance	sexe	categorie	catUp	societe	noEquipe
2	50101	Guillou	Monique	14.04.2013	F		6 true	Lancy	
3	75219	Courtois	Alex	14.08.2004	F		6 false	Pregny-Chambésy	1
4	47718	Le Goff	Véronique	04.06.2015	F		2 true	Pregny-Chambésy	
5	10214	Colin	Benjamin	28.02.2000	M		H true	Versoix	2

Les numéros d'équipe sont assignés par société. Ainsi, chaque société a un numéro 1,2,3, etc. Si aucun numéro d'équipe n'est inscrit, le gymnaste ne fait partie d'aucune équipe.

Ressources Additionnelles

En plus de ce manuel du développeur, il existe d'autres ressources qui peuvent être pertinentes pour compléter votre compréhension du projet et pour vous aider dans votre travail de développement. Voici quelques suggestions :

- **Tests** : Le document sur les tests fournit une documentation complète de tous les tests de l'application, y compris les tests unitaires et les tests d'intégration. Il explique en détail les différents cas de tests et les outils utilisés.
- **Protection des données** : Ce document aborde toutes les questions liées à la protection des données, y compris la collecte, le stockage et l'utilisation des données des utilisateurs. Il décrit les mesures de sécurité mises en place pour garantir la confidentialité et l'intégrité des données, ainsi que les réglementations de protection des données applicables.
- **Maintenance et Sécurité** : Le document sur la maintenance et la sécurité fournit des instructions sur la manière de maintenir et de sécuriser l'application. En ce qui concerne la maintenance, il explique comment gérer Firebase et son utilisation des quotas, en surveillant les limites de lecture et d'écriture pour garantir que l'application reste opérationnelle sans dépasser les limites gratuites du plan Spark. En termes de sécurité, il détaille également les différents rôles et autorisations définis dans Firebase pour contrôler l'accès aux données, ainsi que les méthodes d'authentification des utilisateurs, notamment l'authentification par e-mail et mot de passe.
- **Transition des données** : Ce document explique comment importer les données dans la base de données Firebase. Il fournit des instructions détaillées sur l'utilisation des fichiers conçus pour insérer efficacement les données des sociétés et des gymnastes dans Firestore.

Pour installer Node.js : <https://nodejs.org/en/download>

Pour accéder à la console Firebase, suivre ce lien et cliquer sur "Go to console" :

https://firebase.google.com/?gad_source=1&gclid=Cj0KCQjwudexBhDKARIsAI-GWYVTKsDj5gR2vFEVBXaUucl_kb8jFwP0Mm8sUOvSrbvuEV2k2U80mDQaAoX8EALw_wcB&gclsrc=aw.ds

Documentation de Firebase : <https://firebase.google.com/docs/build?hl=fr>

Documentation de React Query (TanStack Query) :

<https://tanstack.com/query/latest/docs/framework/react/overview>

Liste des annexes

Dossier *PSM_ManuelDeveloppeur_Annexes* :

- Import CSV :

PSM_Annexe_Import.csv

PSM_Annexe_ImportEquipe.csv

- Export CSV :

PSM_Annexe_Export.csv

PSM_Annexe_ExportEquipe.csv